KryptoBirdz

NFT Marketplace — Full Technical Documentation

Version	1.0
Token Standard	ERC-721 (Custom Implementation)
Solidity Version	^0.8.0
Framework	Truffle 5.0.5 / React 16 / Web3.js
Network	Ethereum (Ganache / Mainnet-ready)
Author	clarionnorth@gmail.com

Table of Contents

SECTION 01

Project Overview

KryptoBirdz is a fully on-chain NFT marketplace built from scratch on Ethereum. It implements the ERC-721 standard without any third-party libraries — every interface and base contract is handwritten, providing a deep understanding of how the token standard works under the hood.

What It Does

- Mint unique NFTs on-chain via MetaMask
- Store token URIs referencing bird images
- Display minted tokens in a React gallery
- Track ownership, prevent duplicate minting
- Support ERC-721 transfer and enumeration

Technology Stack

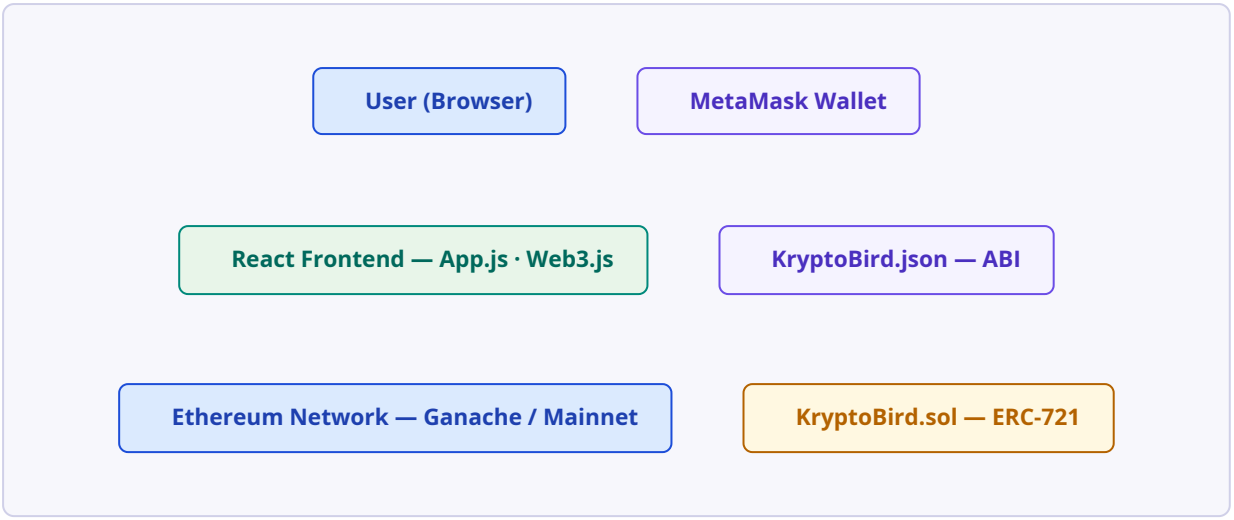
- **Blockchain:** Ethereum / Ganache
- **Contracts:** Solidity ^0.8.0
- **Framework:** Truffle 5.0.5
- **Frontend:** React 16 + Web3.js
- **Wallet:** MetaMask
- **Testing:** Mocha + Chai

Attribute	Value
Collection Name	KryptoBird
Token Symbol	KBIRDZ
Total Unique NFTs	20 uniquely generated pieces
Token Standard	ERC-721 (custom implementation)
Solidity Version	^0.8.0
Network	Ethereum — Ganache development / mainnet-ready
Package Manager	npm
Author	clarionnorth@gmail.com

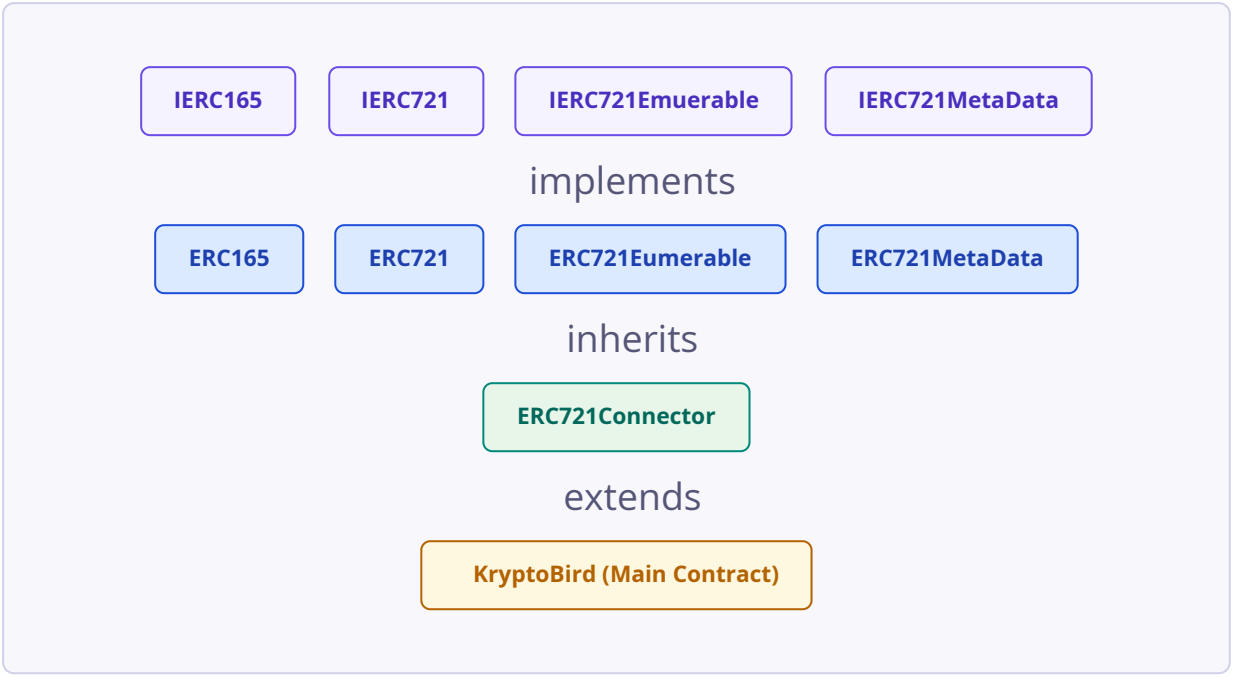
SECTION 02

Architecture & Project Structure

System Architecture



Contract Inheritance Chain



Directory Structure

nft-marketplace/ **src/** **abis/** *compiled ABI + addresses* KryptoBird.**json** **components/** App.**js** *main React component* App.**css** **contracts/** Migrations.**sol** ERC165.**sol** ERC721.**sol** ERC721Eumerable.**sol** ERC721MetaData.**sol** ERC721Connector.**sol** KryptoBird.**sol** **interfaces/** IERC165.**sol** IERC721.**sol** IERC721Emuerable.**sol** IERC721MetaData.**sol** index.**js** *React entry point*

```
serviceWorker.js    migrations/    1_initial_migration.js    2_deploy_contracts.js    test/
KryptoBird.test.js    truffle-config.js    package.json
```

SECTION 03

Smart Contract Layer Overview

The smart contract layer follows a clean inheritance hierarchy mirroring the official ERC-721 specification. Every layer adds specific functionality and registers its interface using EIP-165 introspection.

Contract / Interface	Role	Key Responsibility
`IERC165`	Interface	Defines `supportsInterface()` for on-chain interface detection
`IERC721`	Interface	Core NFT: `balanceOf`, `ownerOf`, `transferFrom`, Transfer event
`IERC721Emuerable`	Interface	`totalSupply`, `tokenByIndex`, `tokenOfOwnerByIndex`
`IERC721MetaData`	Interface	Collection `name()` and token `symbol()`
`ERC165`	Base	Interface registry via `_supportedInterfaces` mapping
`ERC721`	Base	Token storage, minting, ownership mappings, transfer logic
`ERC721Eumerable`	Base	All-tokens array, per-owner token lists, index mappings
`ERC721MetaData`	Base	Stores and exposes `_name` and `_symbol`
`ERC721Connector`	Connector	Combines MetaData + Enumerable into one deployable base
`KryptoBird`	Main	`mint()`, duplicate guard, public `kryptoBirdz[]` array
`Migrations`	Utility	Truffle migrations tracker, owner-only access

SECTION 04

Interfaces — ERC Standards

4.1 IERC165.sol

The foundational interface-introspection standard. Any contract that needs to advertise supported interfaces must implement `supportsInterface`.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IERC165 {
    // Query if a contract implements an interface
    function supportsInterface(bytes4 interfaceID) external view returns (bool);
}
```

4.2 IERC721.sol

The core NFT standard. Defines token ownership queries, transfer capability, and the `Transfer` event emitted on every ownership change.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IERC721 {
    // Emitted when ownership of any NFT changes
    event Transfer(
        address indexed _from,
        address indexed _to,
        uint256 indexed _tokenId
    );

    function balanceOf(address _owner) external view returns (uint256);
    function ownerOf(uint256 _tokenId) external view returns (address);
    function transferFrom(address _from, address _to, uint256 _tokenId) external;
}
```

4.3 IERC721Emuerable.sol

Adds enumeration capability — allows callers to discover all token IDs or all tokens held by a specific address without maintaining off-chain state.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IERC721Emuerable {
    function totalSupply() external view returns (uint256);
    function tokenByIndex(uint256 _index) external view returns (uint256);
    function tokenOfOwnerByIndex(address _owner, uint256 _index)
        external view returns (uint256);
}
```

4.4 IERC721MetaData.sol

Provides human-readable collection identity: a full name and a short symbol, analogous to ERC-20 token metadata.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IERC721MetaData {
    function name() external view returns (string memory _name);
    function symbol() external view returns (string memory _symbol);
}
```

SECTION 05

Core Contracts

5.1 ERC165.sol — Interface Registry

Maintains a private `mapping(bytes4 => bool)` tracking which interface IDs the contract supports. Each contract in the hierarchy calls `_registerInterface` in its constructor with an XOR-combined function selector.

> **Interface ID Computation:** An interface ID is the XOR of all function selectors in that interface. Each selector is the first 4 bytes of the keccak256 hash of the function signature string.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
import './interfaces/IERC165.sol';

contract ERC165 is IERC165 {

  mapping(bytes4 => bool) private _supportedInterfaces;

  constructor() {
    _registerInterface(bytes4(keccak256('supportsInterface')));
  }

  function supportsInterface(bytes4 interfaceID)
    external view override returns (bool) {
    return _supportedInterfaces[interfaceID];
  }

  function _registerInterface(bytes4 interfaceId) internal {
    require(interfaceId != 0xffffffff, 'Invalid interface request');
    _supportedInterfaces[interfaceId] = true;
  }
}
```

5.2 ERC721.sol — Core Token Logic

The heart of the NFT standard. Manages three critical mappings and provides the foundational `_mint` and `_transferFrom` internal functions.

Storage Variable	Type	Purpose
`_tokenOwner`	`mapping(uint => address)`	Maps each token ID to its owner's address
`_OwnedTokensCount`	`mapping(address => uint)`	Tracks how many tokens each address holds
`_tokenApprovals`	`mapping(uint256 => address)`	Reserved for approved transfer addresses

Function	Visibility	Description
`balanceOf(address)`	**public view**	Returns token count for owner. Reverts on zero address.
`ownerOf(uint256)`	**public view**	Returns owner address. Reverts if token non-existent.
`_exists(uint256)`	**internal view**	Returns true if token is minted (owner ≠ address(0)).
`_mint(address, uint256)`	**internal virtual**	Mints token. Validates non-zero address and unique ID. Emits Transfer from address(0).
`_transferFrom(...)`	**internal**	Updates counts and owner mapping. Emits Transfer event.
`transferFrom(...)`	**public override**	Public wrapper calling `_transferFrom`.

```
function _mint(address to, uint256 tokenId) internal virtual {
    require(to != address(0),    'ERC721: minting to non zero address');
    require(!_exists(tokenId),   'ERC721: token already minted');
    _tokenOwner[tokenId]    = to;
    _OwnedTokensCount[to] += 1;
    emit Transfer(address(0), to, tokenId);
}
```

5.3 ERC721Eumerable.sol — Enumeration Extension

Extends ERC721 with four data structures maintaining global and per-owner ordered token lists.

Variable	Type	Purpose
`_allTokens`	`uint256[]`	Ordered list of every minted token ID
`__allTokensIndex`	`mapping(uint256 => uint256)`	Token ID position in _allTokens
`_ownedTokens`	`mapping(address => uint256[])`	Per-owner list of token IDs
`_ownedTokensIndex`	`mapping(uint256 => uint256)`	Token ID index within owner's list

Override Pattern: `_mint` is overridden here using `override(ERC721)` to call `super._mint` first, then push the new token ID into both the global and owner-specific enumeration arrays.

5.4 ERC721MetaData.sol — Collection Identity

Stores collection name and symbol as private strings, initialized once in the constructor and exposed via read-only getters.

```
constructor(string memory named, string memory symbolified) {
    _registerInterface(
        bytes4(keccak256('name(bytes4)')) ^
        bytes4(keccak256('symbol(bytes4)'))
    );
    _name   = named;
    _symbol = symbolified;
}
```

5.5 ERC721Connector.sol — Composition Bridge

A thin glue contract inheriting from both `ERC721MetaData` and `ERC721Eumerable`, forwarding constructor arguments. `KryptoBird` inherits from this single contract to get the full feature set.

```
contract ERC721Connector is ERC721MetaData, ERC721Eumerable {
    constructor(string memory name, string memory symbol)
        ERC721MetaData(name, symbol) {}
}
```

SECTION 06

KryptoBird Main Contract

The final deployable contract tying together the full ERC-721 implementation with KryptoBirdz-specific minting logic.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
import './ERC721Connector.sol';

contract KryptoBird is ERC721Connector {

    // Stores all NFT URIs; index = token ID
    string[] public kryptoBirdz;

    // Prevents minting the same URI twice
    mapping(string => bool) _kryptoBirdzExists;

    function mint(string memory _kryptoBird) public {
        require(!_kryptoBirdzExists[_kryptoBird],
            'Error- KryptoBird already exists');

        kryptoBirdz.push(_kryptoBird);
        uint _id = kryptoBirdz.length - 1;
        _mint(msg.sender, _id);
        _kryptoBirdzExists[_kryptoBird] = true;
    }

    constructor() ERC721Connector('KryptoBird', 'KBIRDZ') {}
}
```

Mint Flow — Step by Step

1. User submits a URI string (e.g. an image URL) via the React form.
2. `_kryptoBirdzExists[URI]` is checked — transaction reverts if already minted.
3. URI is pushed to `kryptoBirdz[]`; array index becomes the token ID (0, 1, 2, ...).
4. `_mint(msg.sender, _id)` is called — updates ownership mappings and emits `Transfer(address(0), msg.sender, _id)`.
5. URI is flagged `true` in the duplicate guard mapping.

> **Design Note:** Token IDs are derived from array position, so they are strictly sequential and gaps form if tokens could ever be burned. This design is intentional for this collection.

SECTION 07

Migrations & Deployment

7.1 Migrations.sol

Standard Truffle migrations contract. Tracks completed migrations to prevent re-deployment, using an owner pattern with the `restricted` modifier.

```
contract Migrations {
  address public owner;
  uint public  last_completed_migration;

  modifier restricted() {
    require(msg.sender == owner, "Restricted to contract owner");
    _;
  }

  constructor() { owner = msg.sender; }

  function setCompleted(uint completed) public restricted {
    last_completed_migration = completed;
  }

  function upgrade(address new_address) public restricted {
    Migrations upgraded = Migrations(new_address);
    upgraded.setCompleted(last_completed_migration);
  }
}
```

7.2 Migration Scripts

1_initial_migration.js

```
const Migrations = artifacts.require("Migrations");
module.exports = function(deployer) {
  deployer.deploy(Migrations);  // Always runs first
};
```

2_deploy_contracts.js

```
const Kryptobird = artifacts.require("Kryptobird");
module.exports = function(deployer) {
  deployer.deploy(Kryptobird);  // Outputs ABI + address to src/abis/
};
```

7.3 Truffle Configuration

```
module.exports = {
  networks: {
    development: {
      host:     "127.0.0.1",
      port:     7545,   // Ganache default port
      network_id: "*",    // Match any network ID
    },
  },
  contracts_directory:      './src/contracts/',
  contracts_build_directory: './src/abis',
  compilers: {
    solc: {
      version:    '^0.8.0',
      optimizer:  { enabled: true, runs: 200 },
      evmVersion: 'london'
    },
  }
};
```

Optimizer Note: `runs: 200` balances deployment gas cost against per-call cost — optimal for contracts called a moderate number of times.

SECTION 08

Frontend Application

8.1 Entry Point — index.js

Standard Create React App entry. Mounts the App component, imports Bootstrap, and leaves the service worker unregistered.

```
import React      from 'react';
import ReactDOM   from 'react-dom';
import            'bootstrap/dist/css/bootstrap.css';
import App        from './components/App';
import * as serviceWorker from './serviceWorker';

ReactDOM.render(<App />, document.getElementById('root'));
serviceWorker.unregister();
```

8.2 Main Component — App.js

A class-based React component managing the full dApp lifecycle: wallet connection, blockchain data loading, minting, and rendering the NFT gallery.

State Shape

Property	Type	Description
`account`	string	Connected MetaMask wallet address
`contract`	object | null	Web3 contract instance
`totalSupply`	number	Total NFTs minted so far
`kryptoBirdz`	string[]	Token URI strings loaded for gallery display

loadWeb3()

```
async loadWeb3() {
   const provider = await detectEthereumProvider();
   if (provider) {
      window.web3 = new Web3(provider);
   } else {
      console.log('no ethereum wallet detected');
   }
}
```

loadBlockchainData()

```
async loadBlockchainData() {
  const web3 = window.web3;
  await window.ethereum.request({ method: 'eth_requestAccounts' });
  const accounts = await web3.eth.getAccounts();
  this.setState({ account: accounts[0] });

  const networkId   = await web3.eth.net.getId();
  const networkData = KryptoBird.networks[networkId];

  if (networkData) {
    const contract    = new web3.eth.Contract(KryptoBird.abi, networkData.address);
    const totalSupply = await contract.methods.totalSupply().call();
    this.setState({ contract, totalSupply });

    for (let i = 0; i < totalSupply; i++) {
      const kryptoBird = await contract.methods.kryptoBirdz(i).call();
      this.setState({ kryptoBirdz: [...this.state.kryptoBirdz, kryptoBird] });
    }
  } else {
    window.alert('Smart contract not deployed');
  }
}
```

mint()

```
mint = (kryptoBird) => {
  this.state.contract.methods.mint(kryptoBird)
    .send({ from: this.state.account })
    .once('receipt', (receipt) => {
      this.setState({
        kryptoBirdz: [...this.state.kryptoBirdz, kryptoBird]
      });
    });
};
```

8.3 Styling — App.css

Selector	Effect
.token	NFT card: 300px wide, box-shadow, 16px margin, 8px border-radius
img	150px wide, rounded corners, subtle shadow, 10px top margin, 5px padding
.container-filled	Full-width absolute container, centered, top: 15%

SECTION 09

Testing Suite

Uses **Mocha** (via Truffle) as test runner, **Chai** for assertions, and **chai-as-promised** for async rejection testing. A single `before` hook deploys the contract once for all test groups.

Test Group 1 — Deployment

Test Case	Assertion
deploys successfully	Contract address is not empty, undefined, null, or 0x0
has a name	`contract.name()` === `'KryptoBird'`
has a symbol	`contract.symbol()` === `'KBIRDZ'`

Test Group 2 — Minting

Test Case	Assertions
creates a new token	Total supply = 1 after mint. Event `_from` = zero address (indicates a mint). Event `_to` = `accounts[0]`. Re-minting the same URI is **rejected**.

Test Group 3 — Indexing

Test Case	Assertions
lists KryptoBirdz	Mints https...2, https...3, https...4 (https...1 from previous group). Iterates 0 totalSupply-1 building a result array. Asserts result.join(',') === `'https...1,https...2,https...3,https...4'`.

> **Running the tests:** Use `truffle test`. Truffle deploys a fresh contract instance to Ganache before each run, ensuring full test isolation.

Configuration & Dependencies

10.1 Key Dependencies

Package	Version	Purpose
`web3`	1.0.0-beta.55	Ethereum JavaScript API
`@metamask/detect-provider`	^1.2.0	Safe MetaMask / injected wallet detection
`truffle`	5.0.5	Smart contract framework
`react`	16.8.4	UI library
`react-dom`	16.8.4	DOM rendering
`react-scripts`	2.1.3	CRA build toolchain
`mdb-react-ui-kit`	^1.3.0	Material Design Bootstrap components
`bootstrap`	4.3.1	CSS grid & utilities
`chai`	4.2.0	BDD assertion library
`chai-as-promised`	7.1.1	Async / rejection assertions
`@openzeppelin/contracts`	^4.3.1	Available as reference (not directly imported)
`babel-*`	various	ES6+ transpilation for Truffle tests

10.2 npm Scripts

Command	Description
`npm start`	Start dev server at localhost:3000
`npm run build`	Production build output into /build
`npm test`	Run Jest tests in watch mode
`truffle test`	Run Mocha smart contract tests
`truffle migrate --reset`	Deploy contracts to configured network
`truffle compile`	Compile Solidity and output ABI JSON

SECTION 11

NFT Collection

The KryptoBirdz collection consists of **11 unique bird NFTs** hosted externally and referenced via URL in the smart contract. Minting any of these URLs registers permanent on-chain ownership.

Token ID	Name	Image URL
0	K1	https://i.ibb.co/9kzVtQy2/K1.png
1	K2	https://i.ibb.co/84mY5WLT/K2.png
2	K3	https://i.ibb.co/PZ3w82tJ/K3.png
3	K4	https://i.ibb.co/1gt4bjG/K4.png
4	K5	https://i.ibb.co/VWL8ZMYX/K5.png
5	K6	https://i.ibb.co/Zpt5B4BP/K6.png
6	K7	https://i.ibb.co/gbDhPH6F/K7.png
7	K8	https://i.ibb.co/vxgYzSW5/K8.png
8	K9	https://i.ibb.co/rKzthRGz/K9.png
9	K10	https://i.ibb.co/xv11Rx5/K10.png
10	K11	https://i.ibb.co/3mQYFbq7/K11.png

Storage Note: Token URIs point to ImgBB-hosted images. For production NFTs, pin assets to **IPFS** (via Pinata or nft.storage) for permanent, decentralized availability.

SECTION 12

Setup & Running Guide

Prerequisites

Tool	Version	Purpose
Node.js	≥ 12.x	JavaScript runtime
npm	≥ 6.x	Package manager
Ganache	2.x GUI or CLI	Local Ethereum blockchain on port 7545
MetaMask	Latest	Browser wallet connected to Ganache
Truffle	5.0.5	Contract compilation, migration, testing

Step-by-Step Setup

1. **Clone the repository and install dependencies**

```
git clone <repository-url> nft-marketplace
cd nft-marketplace
npm install
```

2. **Start Ganache** — launch the GUI workspace or run the CLI:

```
ganache-cli --port 7545
```

Ensure it listens on `127.0.0.1:7545` as configured in `truffle-config.js`.

3. **Configure MetaMask** — add a custom network with:
    4. RPC URL: `http://127.0.0.1:7545`
    5. Chain ID: `1337` (Ganache default)
    6. Import a Ganache account using its private key

7. **Compile and deploy contracts**

```
truffle compile
truffle migrate --reset
```

This generates `src/abis/KryptoBird.json` with the ABI and deployed network address.

8. **Run the test suite**

```
truffle test
```

All 5 tests in `KryptoBird.test.js` should pass.

9 **Start the React frontend**

```
npm start
```

Visit `http://localhost:3000`. MetaMask will prompt for connection. Paste any K1–K11 image URL into the mint form and click **MINT**.

> **You're live!** After minting, the NFT card appears in the gallery below the form, displaying the bird image, collection title, and a Download button linked to the token URI.
